

Nom:

Prénom:

Note:

Exercice 1 ►

/2 points

Sans aucune justification, donner le nombre d'états possibles codables avec:

- 1) 1 bit – 2 états
- 2) 3 bits – $2^3 = 8$ états
- 3) 8 bits – $2^8 = 256$ états
- 4) n bits – 2^n états

Exercice 2 ►

/1 point

Convertir le nombre binaire suivant en décimal. Vous justifierez vos réponses.

1010 0011

$$N = 1 \times 2^7 + 0 \times 2^6 + 1 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 128 + 32 + 2 + 1 = 163$$

Exercice 4 ►

/1 point

Convertir le nombre hexadécimal suivant en décimal. Vous justifierez vos réponses.

AF

$$N = 10 \times 16^1 + 15 \times 16^0 = 160 + 15 = 175$$

Exercice 6 ►

/1 point

Effectuer l'addition binaire suivante:

10110111+1101110

$$\begin{array}{r} 1111 \\ 10110111 \\ + 1101110 \\ \hline 100010101 \end{array}$$

Exercice 7 ►

/2 points

Expliquer simplement comment déterminer si un nombre binaire est pair ou impair. Justifier.

Un nombre binaire est pair si son bit de poids faible (le bit le plus à droite) est égal à 0, et impair s'il est égal à 1. En effet, le bit de poids faible correspond à $2^0 = 1$. Si ce bit est à 1, alors le nombre total est impair (car on ajoute 1 au reste du nombre qui est forcément pair). Si ce bit est à 0, alors le nombre total est pair (car on n'ajoute rien au reste du nombre qui est forcément pair).

Exercice 8 ►

/1 point

```
def est_pair(n):
    ''' Renvoie True si n est pair et False sinon
    n chaîne de caractères représentant un nombre binaire
    sortie: bool '''
    longueur = len(n)
    if n[longueur - 1] == '0':
        return True
    else:
        return False
```

Exercice 9 ►

/2 points

```
def nb_bits(n):
    ''' Renvoie le nombre de bits nécessaires pour écrire n en binaire
    n entier naturel (int)
    sortie: int '''
    compteur = 0
    while n > 0:
        n = n // 2
        compteur += 1
    return compteur
```

Exercice 10 ►

/2 points

```
def nb_uns(val_bin):
    ''' Renvoie le nombre de bits à 1 dans val_bin
    val_bin chaîne de caractères représentant un nombre binaire
    sortie: int '''
    compteur = 0
    for bit in val_bin:
        if bit == '1':
            compteur += 1
    return compteur
```

Exercice 11 ►

/1 point

```
def multiplication(val_bin, puissance):
    ''' Multiplie le nombre binaire val_bin par 2^puissance
    val_bin chaîne de caractères représentant un nombre binaire
    puissance entier naturel (int)
    sortie: str '''
    for i in range(puissance):
        val_bin += '0'
    return val_bin
```

Exercice 12 ►

On dispose de deux chaînes binaires A et B de même longueur (8 bits).

On veut produire une chaîne binaire représentant leur somme.

Ecrire une fonction `addition_binaire(A, B)` qui prend en paramètres les deux chaînes binaires A et B et renvoie la chaîne binaire représentant leur somme. On ne pourra pas convertir les valeurs en décimal avant de faire l'addition.

```
def addition_binaire(A, B):
    ''' Renvoie la chaîne binaire représentant la somme de A et B
    A et B chaînes de caractères représentant des nombres binaires de même longueur (8 bits)
    sortie: str '''

    # Initialisation de la retenue et du résultat
    retenue = 0
    resultat = ''

    # Parcours des bits de droite à gauche
    for i in range(len(A)-1, -1, -1):

        # Récupération des bits de A et B
        bit_a = int(A[i])
        bit_b = int(B[i])

        # Calcul de la somme des bits et de la retenue
        somme = bit_a + bit_b + retenue

        # Détermination du bit résultat et de la nouvelle retenue
        if somme >= 2:
            retenue = 1
            somme -= 2
        else:
            retenue = 0

        # Ajout du bit résultat au début de la chaîne résultat
        resultat = str(somme) + resultat

    # Si il reste une retenue, on l'ajoute au début du résultat
    if retenue == 1:
        resultat = '1' + resultat
    return resultat
```

```
assert addition_binaire('00011010', '00000101') == '00011111' # 26 + 5 = 31
assert addition_binaire('11111111', '00000001') == '100000000' # 255 + 1 = 256
assert addition_binaire('10101010', '01010101') == '11111111' # 170 + 85 = 255
```